

充分發揮 Play 帳款服務整合效益

來源：<https://codelabs.developers.google.com/maximise-your-play-billing-integration?hl=zh-tw>

步驟數：9

導入即時開發人員通知 (RTDN) 並使用 Play Billing Lab 進行測試，藉此改善 Play 帳款服務整合作業，提高購買交易的可靠性並減少訂閱者流失，進而提高整體投資報酬率並放心推出應用程式。

目錄

1. [總覽](#)
2. [訂閱和一次性購買的營利策略](#)
3. [設定即時開發人員通知 \(RTDN\)](#)
4. [接收通知](#)
5. [在應用程式的購買流程中附加使用者 ID](#)
6. [授權前請先驗證購買交易](#)
7. [通知 Google 購買交易已處理完畢](#)
8. [使用 Play Billing Lab 測試](#)
9. [恭喜！](#)

1. 總覽

透過 Google Play 提供的這項 Play 帳款服務程式庫整合程式碼研究室，充分提高收益來源，並放心推出應用程式。這個結構化的程式碼研究室會逐步引導您設定、測試及導入可靠的購買交易處理程序，協助您達成[營利目標](#)，並提供更流暢的使用者體驗。

我們會協助您為應用程式和遊戲的訂閱項目和一次性產品，設定[即時開發人員通知 \(RTDN\)](#) 和 [Play Billing Lab](#)。您將瞭解如何降低訂閱者流失率、防範詐欺和濫用行為、測試極端情況、模擬、重現及解決潛在問題，以及實驗優惠和價格異動，同時不影響使用者。

完成課程後，您將能實施重新爭取使用者策略、快速解決整合問題、提高投資報酬率、提供優質體驗，以及放心推出應用程式和更新。

必要條件

- 熟悉基本的 [Play Billing Library 整合](#)
- 熟悉 Android 應用程式開發 (Java)

課程內容

- 如何妥善管理購買生命週期，運用各種技巧提升購物轉換率和客戶回訪率，進而盡量提高成長
- 如何使用 Google Cloud Pub/Sub 設定即時開發人員通知 (RTDN)，然後運用這項功能實施回購活動和其他生命週期管理策略
- 如何在後端伺服器上設定接收器，安全地處理通知，並準確追蹤和授權，以減少非預期的退款或詐欺和濫用風險
- 如何使用 Play Billing Lab 測試整合情形及模擬錯誤，進而提升使用者體驗並降低開發成本

軟硬體需求

- 在 [Google Play 管理中心](#) 中存取應用程式的 Play 開發人員帳戶
 - 存取已啟用 [Google Play Developer API](#) 的 Google Cloud Platform 專案
 - 後端伺服器，用於處理 Android 應用程式的帳戶和授權
 - 在 Play 管理中心為應用程式註冊的[授權測試人員](#)
 - 測試裝置上已安裝 [Play Billing Lab](#)
-

2. 訂閱和一次性購買的營利策略

透過應用程式販售數位產品時，成功的營利策略必須通盤考量使用者體驗，包括一次性購買和訂閱。順暢的體驗可提高購買意願並減少流失。

一次性購買或訂閱項目的一般[購買流程](#)會包含多個階段：

1. 使用者瀏覽要購買的商品。
2. 啟動購買流程，讓使用者完成購買和付款。
3. 將完成的購買交易通知伺服器
4. 在伺服器上驗證購買交易。
5. 將內容提供給使用者。
6. 確認提交內容。如果是消費性產品，請在適當時間消耗購買項目，讓使用者可以再次購買該商品。

您可以透過應用程式內整合，啟動購買流程及管理使用者體驗，但後端必須能掌握使用者購買授權的最新狀況。如要追蹤購買交易，及管理跨平台授權等使用者體驗的其他面向，這一點至關重要。

即時開發人員通知 (RTDN) 是瞭解[購買交易生命週期](#)不同階段的絕佳方式，可有效做為即時成效追蹤工具，以及啟用訂閱者回流策略的工具。

舉例來說，假設使用者剛購買新商品，或剛錯過付款時間，因此訂閱方案進入寬限期。有了適當的 RTDN，您就能近乎即時地掌握使用者狀態的變化，並採取相應行動，例如讓使用者更投入剛購買的項目，或是傳送提醒電子郵件，請他們更新付款詳細資料，以便繼續訂閱。

Truecaller 導入 RTDN 後，成功挽回約 40% 的訂閱者。[操作說明](#)

此外，RTDN 也是新增伺服器端控制項的好方法，即使使用者用戶端發生問題，您也能管理購買交易。舉例來說，假設使用者成功完成購買交易且收到 Google 的確認通知，但使用者的裝置在透過購買事件監聽器收到購買通知前失去網路連線。透過 RTDN，您會透過伺服器收到獨立通知，因此可以辨識購買交易，並授予使用者授權，不必理會用戶端問題，確保購買程序可靠。

如要進一步瞭解目前支援的所有 RTDN 類型，請參閱[這篇文章](#)。每種 RTDN 訊號都代表不同的購買狀態。請務必實作相應的處理機制，確保在您的用途中視需要進行適當處理。本程式碼研究室將逐步說明如何處理安全後端伺服器中的 RTDN 訊息，包括在使用者於應用程式中成功完成購買交易時，接收訊息、驗證購買交易，以及將授權授予正確的使用者。接著，我們將說明如何為應用程式設定 RTDN。

3. 設定即時開發人員通知 (RTDN)

即時開發人員通知 (RTDN) 採用 [Google Cloud Pub/Sub](#)，可讓您對購買交易狀態變更立即做出回應。Cloud Pub/Sub 是一項全代管的即時訊息服務，可在不同應用程式之間收發訊息。Google Play 使用了 Cloud Pub/Sub，會根據訂閱主題發布推播通知。

如要啟用 RTDN，您必須先使用自己的 Google Cloud Platform (GCP) 專案設定 Cloud Pub/Sub，然後為應用程式開啟通知功能。如果不熟悉 GCP 和 Cloud Pub/Sub，請參閱[快速入門導覽課程](#)。

建立主題

如要開始接收通知，您必須建立一個 Google Play 應發布通知的[主題](#)。請按照「[建立主題](#)」中的指示建立主題。

建立 Pub/Sub 訂閱項目

如要接收發布至某項主題的訊息，您必須為該主題建立一個 Pub/Sub 訂閱項目。如要建立 Pub/Sub 訂閱項目，請按照下列步驟操作：

1. 請參閱 [Cloud Pub/Sub 訂閱者指南](#)，瞭解如何將訂閱項目設定為[推送訂閱項目](#)或[提取訂閱項目](#)。在本程式碼研究室中，我們將使用提取式訂閱項目，這需要透過安全的後端伺服器，向 Cloud Pub/Sub 伺服器發出要求來擷取訊息。

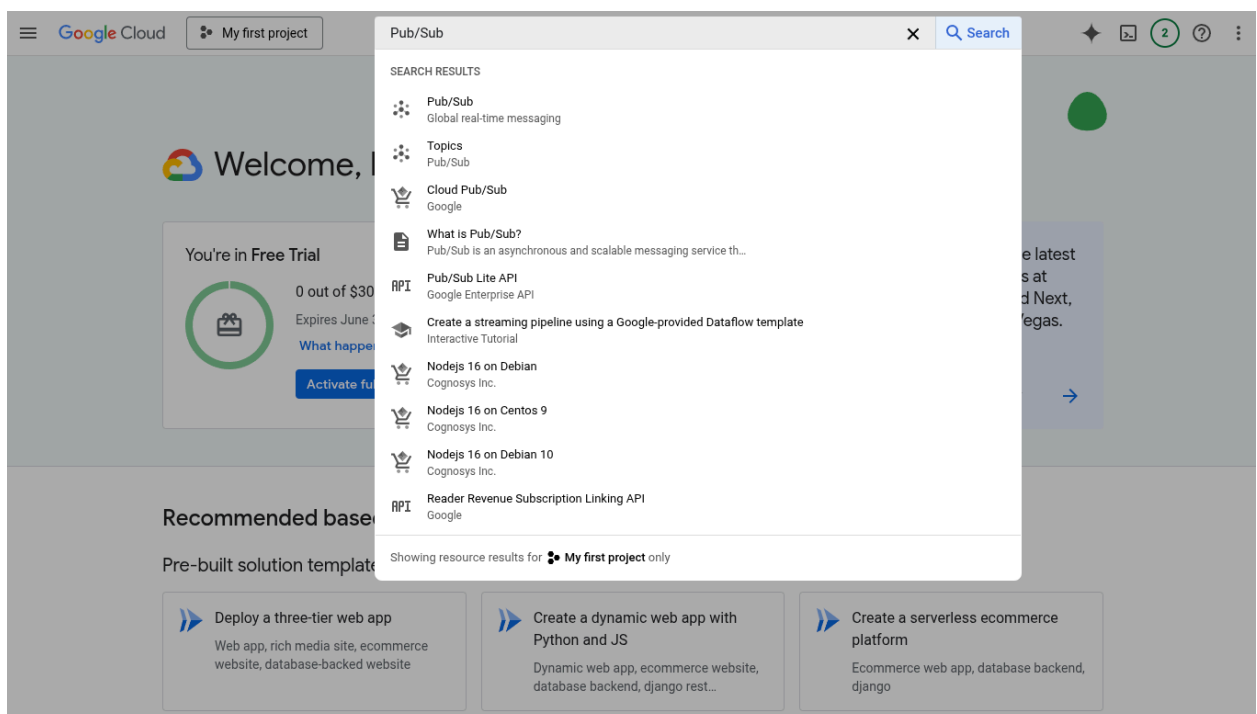
請參閱「[提取式或推送式傳遞](#)」一文，判斷應用程式適合採用推送式或提取訂閱項目。

2. 按照「[新增訂閱項目](#)」中的指示建立訂閱項目。

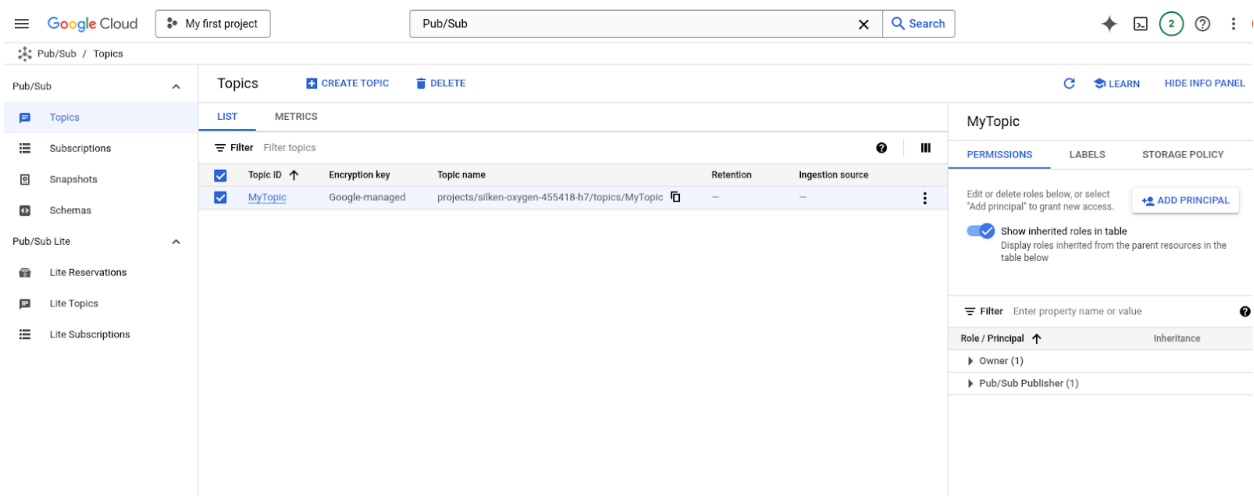
授予主題的發布權限

您必須授予 Google Play 權限，才能讓 [Cloud Pub/Sub](#) 針對您的主題發布通知。

1. 開啟 [Google Cloud 控制台](#)。
2. 選取專案，然後在搜尋列中搜尋「Pub/Sub」，並前往 Pub/Sub 設定頁面。



3. 尋找您的主題，然後開啟權限設定。



4. 按一下「新增主體」，新增服務帳戶 `google-play-developer-notifications@system.gserviceaccount.com`，並授予「Pub/Sub 發布者」角色。

Grant access to "MyTopic"

Grant principals access to this resource and add roles to specify what actions the principals can take. Optionally, add conditions to grant access to principals only when a specific criteria is met. [Learn more about IAM conditions](#)

Resource

 MyTopic

Add principals

Principals are users, groups, domains, or service accounts. [Learn more about principals in IAM](#)

New principals *

?

Assign roles

Roles are composed of sets of permissions and determine what the principal can do with this resource. [Learn more](#)

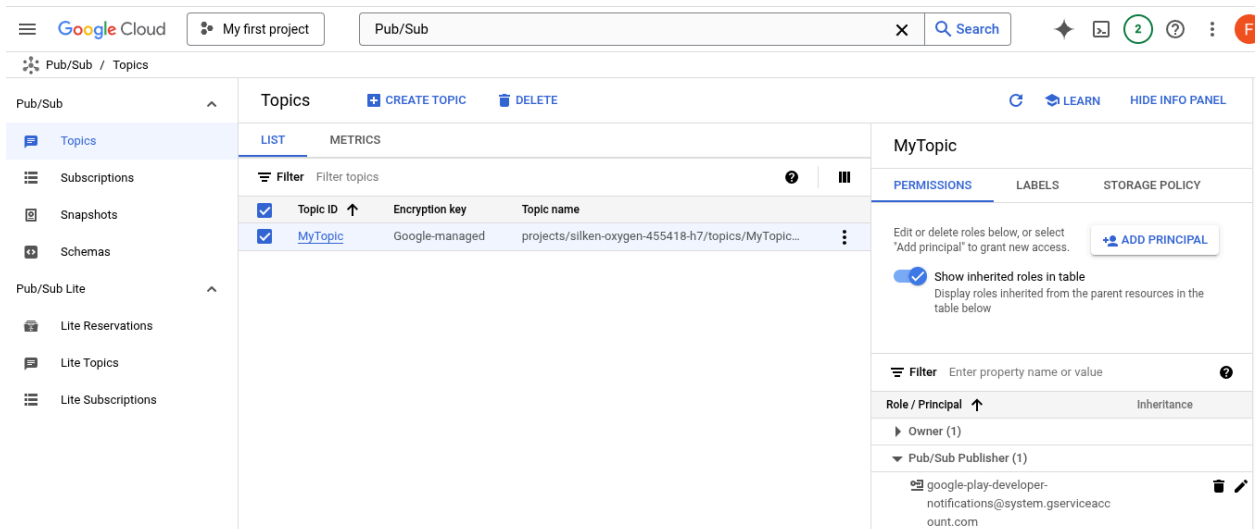
Role *

?

Publish messages to a topic.

[+ ADD ANOTHER ROLE](#)

5. 按一下「儲存」即可完成主題設定。



The screenshot shows the Google Cloud IAM console interface. The left sidebar contains navigation links for 'Pub/Sub / Topics', 'Topics', 'Subscriptions', 'Snapshots', 'Schemas', 'Pub/Sub Lite', 'Lite Reservations', 'Lite Topics', and 'Lite Subscriptions'. The main content area is titled 'Topics' and shows a table with columns 'Topic ID', 'Encryption key', and 'Topic name'. The table contains one entry: 'MyTopic' with a 'Google-managed' encryption key and a topic name 'projects/silken-oxygen-455418-h7/topics/MyTopic...'. The right sidebar shows the 'MyTopic' resource details, including the 'PERMISSIONS' tab, a list of roles/principals, and an 'ADD PRINCIPAL' button. The list of roles/principals shows 'Owner (1)' and 'Pub/Sub Publisher (1)'. The 'Pub/Sub Publisher' role is expanded, showing the principal 'google-play-developer-notifications@system.gserviceaccount.com'.

為應用程式啟用即時開發人員通知

瞭解如何設定即時開發人員通知 (RTDN)，大幅提升 Play 帳款服務整合效果。您可以透過個人化訊息提高購買可靠性，並防範詐欺和濫用行為，進而提高整體投資報酬率。

RTDN 可直接從 Google Play 提供伺服器對伺服器的即時更新，以掌握訂閱續訂、新購買交易和付款問題等重要事件。這類通知可協助後端系統自動與使用者授權的實際狀態同步，突破用戶端限制，讓您即時做出適當反應。

如何為應用程式啟用即時開發人員通知：

1. 開啟 [Google Play 管理中心](#)。
 2. 選取應用程式。
 3. 依序前往「透過 Google Play 營利」>「營利設定」。
 4. 捲動至「即時開發人員通知」部分。
 5. 勾選「啟用即時通知」。
 6. 在「主題名稱」欄位中完整輸入您先前設定的 Cloud Pub/Sub 主題名稱。主題名稱應採用 `projects/{project_id}/topics/{topic_name}` 的格式，其中 `project_id` 是專案的專屬 ID，`topic_name` 是稍早建立的主題名稱。
 7. 按一下「傳送測試訊息」送出測試訊息。測試發布有助於確保已正確完成各項設定。如果測試發布成功，系統會顯示一則訊息，說明測試發布成功。如果這個主題附加了一個訂閱項目，則您應收到該測試訊息。如果是提取訂閱項目，請前往 Cloud 控制台中的訂閱項目，按一下「查看訊息」，然後繼續提取訊息。您應確認接收到的任何訊息，以避免出現 Cloud Pub/Sub 重複傳遞的情況。如果是[推送訂閱項目](#)，請確認測試訊息是否已傳遞至推送端點。成功的回應代碼可用來確認訊息。如果發布失敗，系統會顯示錯誤。請確認主題名稱正確無誤，而且該 `google-play-developer-notifications@system.gserviceaccount.com` 服務帳戶擁有該主題的 **Pub/Sub 發布者** 存取權。
 8. 選擇要接收的通知類型。
- **接收訂閱項目和所有作廢交易的通知：**接收與訂閱項目和作廢交易相關的即時開發人員通知。你不會收到單次產品購買交易的通知。
 - **接收訂閱項目和一次性產品的所有通知：**接收所有訂閱項目和已作廢交易事件的通知。您也會收到單次產品購買事件，例如 `ONE_TIME_PRODUCT_PURCHASED` 和 `ONE_TIME_PRODUCT_CANCELED`。如要進一步瞭解這些購買事件，請參閱「[單次購買生命週期](#)」。

9. 按一下 [儲存變更]。

您已為應用程式完成即時開發人員通知設定，現在可以運用這些工具解決常見問題，例如透過挽回訊息減少使用者流失，或是防範詐欺和濫用行為。在下一節中，我們將在安全後端伺服器中建立訂閱端，以取用傳送給 Cloud Pub/Sub 主題的訊息。

4. 接收通知

請務必讓後端伺服器掌握最新購買交易狀態，確保應用程式提供最佳使用者體驗。舉例來說，使用者在應用程式中成功完成付款後，應盡快將內容傳送至帳戶。

這需要系統及時偵測並處理完成的購買交易。Play 帳款服務程式庫提供多種方式，可[偵測應用程式中的購買交易](#)。偵測到完成的購買交易後，應用程式必須通知後端伺服器驗證交易，將內容授予正確的使用者，然後通知 Google 交易已處理完畢。不過，應用程式可能會因為各種原因，無法及時偵測到購買交易。舉例來說，使用者成功完成購買交易且收到 Google 的確認通知，但使用者的裝置在透過 Play Billing Library 介面收到通知前失去網路連線。即使使用者用戶端發生問題，RTDN 也能提供額外的伺服器端控制項，協助您管理購買交易。RTDN 可確保在購買狀態變更時，伺服器會收到獨立通知，讓您透過第二條路徑幾乎立即辨識購買狀態變更，不受潛在用戶端問題影響，確保購買流程更加可靠。

在本節中，您將使用 [Cloud Pub/Sub 用戶端程式庫](#)，建立訂閱端來取用傳送至 Cloud Pub/Sub 主題的訊息。這些程式庫提供多種語言。在接下來的章節中，我們會新增訂閱者來驗證購買交易、授予正確使用者授權，並在伺服器上確認/消耗購買交易。在本程式碼研究室中，我們使用 Java。

每次發布至 Cloud Pub/Sub 主題的發布內容都會包含一個 Base64 編碼的資料欄位。

```
{  
  "message": {  
    "attributes": {  
      "key": "value"  
    },  
    "data":  
    "eyJhdmVyc2lubiI6IHNN0cmZyYwInBhY2thZ2VOYWllIjogc3RyaW5nLCAiZXZlbmRUaW1lTWlsbG1zIjogbG9uZywgIm9uZVRpbWVQcm9kdWN0Tm90aWZpY2F0aW9uIjogT25lVGltZVB5b2RlY3ROb3RpZmljYXRpb24sICJzdWJzY3JpcHRpb25Ob3RpZmljYXRpb24iOiBTdWJzY3JpcHRpb25Ob3RpZmljYXRpb24sICJ0ZXN0Tm90aWZpY2F0aW9uIjogVGZvdE5vdGlmaWNhdGlvbiB9",  
    "messageId": "136969346945"  
  },  
  "subscription": "projects/myproject/subscriptions/mysubscription"  
}
```

將 Base64 編碼的資料欄位解碼後，DeveloperNotification 會包含以下欄位：

```
{
  "version": string,
  "packageName": string,
  "eventTimeMillis": long,
  "oneTimeProductNotification": OneTimeProductNotification,
  "subscriptionNotification": SubscriptionNotification,
  "voidedPurchaseNotification": VoidedPurchaseNotification,
  "testNotification": TestNotification
}
```

詳情請參閱[即時開發人員通知參考資料](#)。

以下是 NotificationReceiver 的程式碼範例，可供安全後端伺服器處理 Pub/Sub 訊息。如要向 Security Command Center 進行驗證，請設定應用程式預設憑證，詳情請參閱「[設定本機開發環境的驗證機制](#)」。

```

import com.google.cloud.pubsub.v1.AckReplyConsumer;
import com.google.cloud.pubsub.v1.MessageReceiver;
import com.google.cloud.pubsub.v1.Subscriber;
import com.google.pubsub.v1.ProjectSubscriptionName;
import com.google.pubsub.v1.PubsubMessage;
import java.util.Base64;
import org.json.JSONObject;

/** Real-time developer notifications receiver. */
public class NotificationReceiver {

    private NotificationReceiver() {}

    /**
     * Receive notification messages from the subscription.
     *
     * @param projectId The project ID of your Google Cloud Project.
     * @param subscriptionId The subscription ID of the subscriber to the pub/sub topic.
     */
    public static void receiveNotificationMessages(String projectId, String subscriptionId) {
        ProjectSubscriptionName subscriptionName =
            ProjectSubscriptionName.of(projectId, subscriptionId);

        try {
            Subscriber subscriber =
                Subscriber.newBuilder(subscriptionName, new NotificationMessageReceiver()).build();
            // Start the subscriber.
            subscriber.startAsync().awaitRunning();

            subscriber.awaitTerminated();
        } catch (IllegalStateException e) {
            System.out.println("Subscriber stopped: " + e);
        }
    }

    static class NotificationMessageReceiver implements MessageReceiver {

        @Override
        public void receiveMessage(PubsubMessage message, AckReplyConsumer consumer) {
            // Decode the data into a String from the message data field.
            String jsonString = new
String(Base64.getDecoder().decode(message.getData().toStringUtf8()));
            // Parse the String into a JSON object.
            JSONObject messageJson = new JSONObject(jsonString);

            // Fetch the value for certain fields.
            String version = messageJson.getString("version");
            String packageName = messageJson.getString("packageName");
            System.out.println("version: " + version);
            System.out.println("packageName: " + packageName);

            // Validate the purchase and grant the entitlement as needed.

```

```
// More details in the following sections.  
// .....  
  
// Acknowledge the message to avoid repeated delivery.  
consumer.ack();  
}  
}  
}
```

現在，您已在安全後端伺服器中，建立一個通知接收器，可取用傳送至 Cloud Pub/Sub 主題的訊息。在接下來的章節中，我們將介紹在後端伺服器中處理 RTDN 訊息的最佳做法。

5. 在應用程式的購買流程中附加使用者 ID

伺服器收到有關購買狀態更新的 RTDN 訊息時，必須知道是哪位使用者進行購買，才能處理交易，例如將內容傳送給正確的使用者。如要達成這個目標，請在應用程式中啟動購買流程時，使用 `obfuscatedAccountId` 附加您擁有的任何使用者 ID，以識別進行購買交易的使用者。舉例來說，ID 可以是使用者在您系統中的登入資訊經過模糊處理的版本。設定這個參數有助於 Google [偵測詐欺](#)。此外，如「[授予使用者權利](#)」一文所述，這項功能可協助您確保系統將購買交易歸給正確的使用者。

以下顯示範例程式碼，說明如何在應用程式中啟動購買流程時，透過設定 `obfuscatedAccountId` 附加使用者 ID。

```
// An activity reference from which the billing flow will be launched.
Activity activity = ...;
// A user identifier, e.g. an obfuscated user id in your system.
String obfuscatedAccountId = ...;

ImmutableList<ProductDetailsParams> productDetailsParamsList =
    ImmutableList.of(
        ProductDetailsParams.newBuilder()
            // retrieve a value for "productDetails" by calling queryProductDetailsAsync()
            .setProductDetails(productDetails)
            // set the offer token to specify the offer to purchase when applicable, e.g.,
            subscription products
            // .setOfferToken(offerToken)
            .build()
    );

BillingFlowParams billingFlowParams = BillingFlowParams.newBuilder()
    .setProductDetailsParamsList(productDetailsParamsList)
    .setObfuscatedAccountId(obfuscatedAccountId)
    .build();

// Launch the billing flow
BillingResult billingResult = billingClient.launchBillingFlow(activity, billingFlowParams);
```

如下一節所示，在購買流程中設定的使用者 ID 會納入購買交易，並可用於授予正確的使用者權利。

6. 授權前請先驗證購買交易

在本節中，我們將說明在安全後端伺服器中，授權前驗證購買交易的最佳做法。

使用者完成單次產品購買交易後，安全後端伺服器中的 Pub/Sub 訂閱端會收到 Pub/Sub 訊息。您應在後端伺服器中執行下列操作：

1. 剖析 Pub/Sub 訊息中的 `purchaseToken`。您應為所有購買交易維護所有 `purchaseToken` 值的記錄。
2. 確認目前購買交易的 `purchaseToken` 值沒有與之前的任何 `purchaseToken` 值重複。`purchaseToken` 全域唯一，因此您可以放心將這個值做為資料庫中的主鍵。
3. 使用 Google Play Developer API 中的 [purchases.products:get](#) 端點，向 Google 驗證購買交易是否符合規定。

訂閱項目請使用 Google Play Developer API 中的 [purchases.subscriptionsv2:get](#) 端點。

4. 如果應用程式內商品或訂閱項目購買交易合規，且從未使用過，就可以對應用程式內商品或訂閱項目授權。
5. 只有當購買交易狀態為 `PURCHASED` 時，您才應該進行授權，且必須正確處理 `PENDING` 購買交易。詳情請參閱「[處理未完成的交易](#)」。

下列程式碼範例會建立 Google Play Developer API 的 API 用戶端。稍後會用來發出 API 呼叫。

```

import com.google.api.client.auth.oauth2.Credential;
import com.google.api.client.googleapis.auth.oauth2.GoogleCredential;
import com.google.api.client.googleapis.javanet.GoogleNetHttpTransport;
import com.google.api.client.http.HttpTransport;
import com.google.api.client.json.JsonFactory;
import com.google.api.client.json.gson.GsonFactory;
import com.google.api.services.androidpublisher.AndroidPublisher;
import com.google.api.services.androidpublisher.AndroidPublisherScopes;
import java.io.IOException;
import java.security.GeneralSecurityException;
import java.util.Collections;

/** Helper class to initialize the publisher APIs client library. */
public class AndroidPublisherHelper {
    /** Your application name */
    private static final String APPLICATION_NAME = "YourApplicationName";

    /** Load credentials from a JSON key file. Replace with the actual path to your downloaded
    service
    * account key file.
    */
    private static final String RESOURCES_CLIENT_SECRETS_JSON =
        "/path/to/your/service_account_key.json";

    /** Global instance of the JSON factory. */
    private static final JsonFactory JSON_FACTORY = GsonFactory.getDefaultInstance();

    /** The API client */
    private static final AndroidPublisher ANDROID_PUBLISHER = init();

    /**
     * Performs all necessary setup steps for running requests against the API.
     *
     * @return the {@link AndroidPublisher} service
     */
    private static AndroidPublisher init(){
        try {
            // Authorization.
            Credential credential =
                GoogleCredential.fromStream(
                    AndroidPublisherHelper.class.getResourceAsStream(RESOURCES_CLIENT_SECRETS_JSON))
                .createScoped(Collections.singleton(AndroidPublisherScopes.ANDROIDPUBLISHER));

            HttpTransport httpTransport = GoogleNetHttpTransport.newTrustedTransport();

            // Set up and return API client.
            return new AndroidPublisher.Builder(httpTransport, JSON_FACTORY, credential)
                .setApplicationName(ApplicationConfig.APPLICATION_NAME)
                .build();
        } catch (GeneralSecurityException | IOException ex) {
            throw new RuntimeException("fail to initialize the publisher APIs client library", ex);
        }
    }
}

```

```
}  
}  
}
```

接著，我們會新增邏輯來發出 API 呼叫，並修改先前建構的接收器，以驗證購買交易，並將授權授予正確的使用者。

在 `AndroidPublisherHelper` 中，新增下列方法，從 Google Play Developer API 的 [Purchases.products.get](#) 端點擷取 `ProductPurchase`。

```
/* Fetch the ProductPurchase for the one-time product purchase from  
 * Purchases.products.get endpoint in the Google Play Developer API  
 */  
public static ProductPurchase executeProductPurchasesGet(  
    String packageName, String sku, String purchaseToken) {  
    try {  
        ProductPurchase productPurchase =  
            ANDROID_PUBLISHER.purchases().products().get(packageName, sku,  
purchaseToken).execute();  
        return productPurchase;  
    } catch (IOException ex) {  
        log.error("Exception was thrown while getting a product purchase", ex);  
        // It is recommended to apply some retry mechanism, such as exponential backoff, to  
fetch the purchase in case of transient failures.  
        return null;  
    }  
}
```

在 `NotificationMessageReceiver` 中，根據通知中包含的資料驗證購買交易，並在系統中授予正確使用者授權。您應持續追蹤伺服器中的 `purchaseToken`，避免重複處理。

請參閱[即時開發人員通知參考資料](#)，瞭解您可以從 Google Play 收到的所有通知類型，並根據業務需求在伺服器中處理這些通知。


```

@Override
public void receiveMessage(PubsubMessage message, AckReplyConsumer consumer) {
    // Decode the data into a String from the message data field.
    String jsonString = new
String(Base64.getDecoder().decode(message.getData().toStringUtf8()));
    // Parse the String into a JSON object.
    JSONObject messageJson = new JSONObject(jsonString);

    // Fetch the value for certain fields.
    String version = messageJson.getString("version");
    String packageName = messageJson.getString("packageName");

    // Process notification data based on your business requirements.
    // Process oneTimeProductNotification in the message.
    JSONObject oneTimeProductNotificationJson =
        messageJson.getJSONObject("oneTimeProductNotification");
    if (oneTimeProductNotificationJson != null) {
        String purchaseToken = oneTimeProductNotificationJson.getString("purchaseToken");
        String sku = oneTimeProductNotificationJson.getString("sku");
        int notificationType = oneTimeProductNotificationJson.getInt("notificationType");

        if (notificationType == 1) {
            // ONE_TIME_PRODUCT_PURCHASED - A one-time product was successfully purchased by a
user.
            // Verify that the purchaseToken value does not match any previous purchaseToken
values in
            // your backend system to avoid duplicate processing.
            .....
            // Fetch the ProductPurchase from Purchases.products.get endpoint
            ProductPurchase productPurchase =
                AndroidPublisherHelper.executeProductPurchasesGet(packageName, sku, purchaseToken);
            if (productPurchase != null && productPurchase.getPurchaseState() == 0) {
                // The purchase is valid and in PURCHASED state.

                // The account Id set in the App when launching the billing flow.
                String obfuscatedExternalAccountId =
productPurchase.getObfuscatedExternalAccountId();
                // Grant the entitlement to the correct account for obfuscatedExternalAccountId in
your
                // system.
                .....
            }
        }
        // Process subscriptionNotification in the message.
        JSONObject subscriptionNotificationJson =
messageJson.getJSONObject("subscriptionNotification");
        if (subscriptionNotificationJson != null) {
            .....
        }
        // Process other notification data in the message as needed.
        .....
    }
}

```

```
// Acknowledge the message to avoid repeated delivery.  
consumer.ack();  
}
```

7. 通知 Google 購買交易已處理完畢

授權後，您應從安全的後端伺服器呼叫 Play Developer API 中的 [purchases.products:consume](#) 或 [purchases.products:acknowledge](#) 端點，通知 Google 購買交易已處理完畢，以便消耗消耗性產品或確認非消耗性產品。

使用 Google Play Developer API 中的 [purchases.subscriptions:acknowledge](#) 端點確認訂閱項目。

在 `AndroidPublisherHelper` 中，新增下列方法，在 Google Play Developer API 中呼叫 [purchases.products:consume](#) 或 [purchases.products:acknowledge](#)。

```
/* Consume the one-time product purchase by calling
 * Purchases.products.consume endpoint in the Google Play Developer API
 */
public static void executeProductPurchasesConsume(
    String packageName, String sku, String purchaseToken) {
    try {
        ANDROID_PUBLISHER
            .purchases().products().consume(packageName, sku, purchaseToken).execute();
    } catch (IOException ex) {
        log.error("Exception was thrown while consuming a product purchase", ex);
        // It is recommended to apply some retry mechanism, such as exponential backoff, to
        ensure the purchase is correctly consumed in case of transient failures.
    }
}

/* Acknowledge the one-time product purchase by calling
 * Purchases.products.acknowledge endpoint in the Google Play Developer API
 */
public static void executeProductPurchasesAcknowledge(
    String packageName, String sku, String purchaseToken) {
    try {
        ANDROID_PUBLISHER
            .purchases().products().acknowledge(packageName, sku, purchaseToken, new
ProductPurchasesAcknowledgeRequest()).execute();
    } catch (IOException ex) {
        log.error("Exception was thrown while acknowledging a product purchase", ex);
        // It is recommended to apply some retry mechanism, such as exponential backoff, to
        ensure the purchase is correctly acknowledged in case of transient failures.
    }
}
```

在 `NotificationMessageReceiver` 中，於後端伺服器授權後，消耗消耗性產品購買交易或確認非消耗性產品購買交易。

```

@Override
public void receiveMessage(PubsubMessage message, AckReplyConsumer consumer) {
    .....
    String obfuscatedExternalAccountId = productPurchase.getObfuscatedExternalAccountId();
    // Grant the entitlement to the correct account for obfuscatedExternalAccountId in
your
    // system.
    .....
    // If the product is a consumable product, consume the purchase.
    AndroidPublisherHelper.executeProductPurchasesConsume(packageName, sku,
purchaseToken);
    // Or if the product is a non-consumable product, acknowledge the purchase.
    // AndroidPublisherHelper.executeProductPurchasesAcknowledge(packageName, sku,
purchaseToken);
    .....
}

```

確認為必要動作，因為這可通知 Google Play，使用者已取得購買交易的授權。您應在授權後立即確認購買交易。

做得好！您已成功整合即時開發人員通知，可如本程式碼研究室所示，可靠地處理購買交易。現在，為了確保一切運作正常，我們來瞭解 Play Billing Lab。這項工具操作簡單，可協助您測試 Play 帳款服務整合情形。

8. 使用 Play Billing Lab 測試

為確保順利發布，您應在整個開發過程中測試整合。[Play Billing Lab](#) 是一款免費的 Android 應用程式，可協助開發人員測試與 Google Play 帳款服務的整合情形，讓開發人員輕鬆方便地測試 Play 帳款服務功能、加快整合速度，並更有把握地推出應用程式。

Play Billing Lab 提供多種測試功能，協助您測試不同情境，包括：

- 在 Play Billing Lab 中變更 [Play 國家/地區](#)，然後將設定套用至測試。無論測試人員實際在何處進行測試，都能[測試不同國家/地區的自訂使用者體驗](#)
- 使用同一帳戶重複[測試試用或新用戶優惠](#)
- [測試訂閱價格異動](#)，避免影響其他有效訂閱者
- [模擬 Google Play Billing Library 回應代碼](#)，在各種錯誤情境下進行測試
- [加快訂閱續訂時間](#)，加快測試速度
- [使用實際付款方式測試](#)，略過特定購買流程風險信號

我們不斷為 Play Billing Lab 應用程式新增測試功能。您可以從 [Play 商店](#) 下載及安裝 Play Billing Lab，或參閱「[測試整合情形](#)」一文，進一步瞭解如何使用 Play Billing Lab 進行測試。

使用 Play Billing Lab 測試 BillingResponseCode

整合應用程式與 Play 結帳程式庫時，測試所有 [BillingResponseCode](#) 流程是一項常見挑戰，因為您無法有效控管 Play 商店與 Play 後端之間的通訊。您可以使用 Play Billing Lab 應用程式的「回應模擬器」功能，為 Play Billing Library 設定錯誤代碼回應，測試各種複雜的錯誤情境。

舉例來說，您在應用程式中導入邏輯，讓應用程式偵測到交易成功後，即會使用購買交易。您想測試應用程式因網路連線失敗而無法消耗購買交易的情境，並確認後端伺服器中的 RTDN 接收器是否會接收訊息並正確處理購買交易。您可以運用回應模擬器模擬測試情境。以下將逐步說明如何使用 Play Billing Lab 回應模擬器進行測試。

使用回覆模擬器測試

使用回應模擬器測試時，應用程式會與 Play Billing Lab 通訊，以取得您在 Play Billing Lab 回應模擬器中設定的回應代碼。

為 Play Billing Library 啟用帳單覆寫測試

如要啟用回應模擬器與應用程式之間的通訊，請先在應用程式中啟用 Play Billing Library 的帳單覆寫測試。如要執行這項操作，請將下列中繼資料標記新增至應用程式的 `AndroidManifest.xml` 檔案。

```
<manifest ... >
  <application ... >
    ...
    <meta-data
      android:name="com.google.android.play.largest_release_audience.NONPRODUCTION"
      android:value="" />
    <meta-data
      android:name="com.google.android.play.billingclient.enableBillingOverridesTesting"
      android:value="true" />
  </application>
</manifest>
```

使用更新後的 `AndroidManifest.xml` 檔案建構應用程式。現在，您的應用程式已準備好使用 Play Billing Lab 回應模擬器。

即使只新增 `enableBillingOverridesTesting` 標記並設為 `true`，就足以為應用程式啟用帳單覆寫測試，但我們仍建議您也新增 `NONPRODUCTION` 標記。`NONPRODUCTION` 標記可防止您在啟用帳單覆寫測試的情況下，不慎將應用程式部署到正式環境。

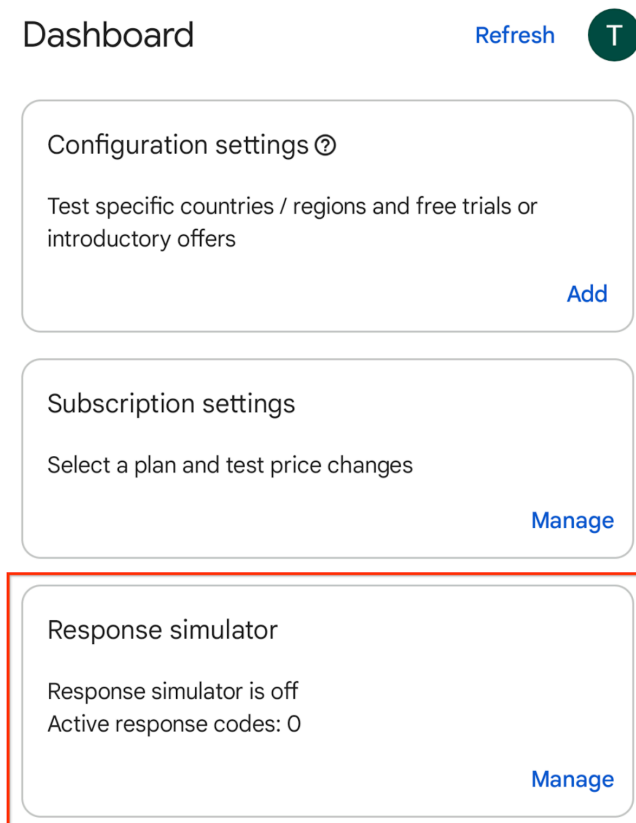
測試完畢後，將應用程式部署至正式環境時，請使用不含這些中繼資料標記的獨立 `AndroidManifest.xml` 檔案，或確保已從 `AndroidManifest.xml` 檔案中移除這些標記。

模擬 Google Play Billing Library 錯誤

如要使用模擬的 Play Billing Library 錯誤進行測試，請先在 Play Billing Lab 應用程式中設定回應代碼，然後在應用程式中執行測試。

設定回應代碼

1. 使用應用程式的[授權測試人員](#)帳戶登入 Play Billing Lab 應用程式。下圖顯示 Play Billing Lab 的資訊主頁，包括「回應模擬器」資訊卡。



2. 按一下「回應模擬器」資訊卡上的「管理」，即可前往「回應模擬器」畫面。
3. 出現提示時，請允許 Play Billing Lab 傳送通知，以便查看應用程式的連線狀態。
4. 如果尚未啟用「模擬 Google Play Billing Library 回應」切換鈕，請啟用。

← Response simulator

Simulate Play Billing Library responses
Simulate errors using the Play Billing Library response codes. [Learn more](#)



Active response codes: 0

launchBillingFlow

Select response code ▼

queryProductDetailsAsync

Select response code ▼

consumeAsync

Select response code ▼

acknowledgePurchase

Select response code ▼

querySkuDetailsAsync

Select response code ▼

5. 選取要測試的 Play Billing Library API 回應代碼。如要模擬消耗購買交易時發生錯誤，請為 `consumeAsync` API 選取錯誤代碼。系統會自動儲存所選項目。現在回應模擬器已準備好將所選回應代碼傳送至應用程式。

測試應用程式

現在您可以測試應用程式，確認在設定的錯誤情境中，一切是否正常運作。開啟應用程式並觸發 Play 結帳服務程式庫 API 方法。如果應用程式發出 `consumeAsync` 呼叫來消耗購買交易，應用程式就會收到您剛才設定的錯誤代碼。您可以根據錯誤代碼確認應用程式是否正常運作，以及後端伺服器是否正確處理購買交易。

測試完成後，只要關閉「模擬 Google Play Billing Library 回應」切換鈕，即可停止模擬回應。

如要進一步瞭解如何使用 [Play Billing Lab](#) 進行測試，請參閱說明中心文章，瞭解如何[使用授權測試人員測試應用程式內結帳功能](#)。

9. 恭喜！

您已完成本程式碼研究室，現在有能力策略性地最佳化應用程式營利，改善使用者體驗，進而提升使用者滿意度、購物轉換和訂閱者流失率。

善用即時開發人員通知和 [Play Billing Lab](#) 隨附應用程式，主動處理[單次購買](#)和[訂閱](#)的購買交易生命週期事件。

有了這些工具，您就能有效實施吸引人的回訪策略、迅速解決整合問題，並最終提升使用者體驗和收益來源，有把握地推出應用程式或遊戲。

完成這個程式碼研究室後，您已具備管理完整購物歷程的技能，並能使用 Play Billing Lab 嚴格測試實作項目，確保使用者體驗流暢，並在 Google Play 上盡可能提高營利。
